Rule-Based Alerting on Binance Market Data

Python for Engineers
FA26HN MSA42HN
Group 02

Lecturer Bùi Trung Hiếu
15 October 2026

Background and user needs

An individual user follows several market pairs while working or studying.

Manual chart checks and copying values into a chatbot interrupt that workflow.

The application stores conditions, reports supported matches through Telegram, and explains the supplied data.

Objectives and measured progress

Objective	Observed evidence	Status
Five market pairs	300 candles returned for each pair	On demand
Five indicator families	SMA, EMA and RSI implemented	3 of 5
Rule and notification workflow	Redis CRUD and one accepted Telegram send	Delivered subset
AI evaluation	Six live questions and recorded scores	Exploratory pilot

No profitability or forecast-accuracy claim forms part of acceptance.

Task allocation and contribution

Member	Primary responsibility	Share %
Phương Hữu Thành Nam	Indicator formulas and numerical fixtures	25
Bùi Quý Đăng	OpenAI and Telegram integration	25
Nguyễn Anh Vũ	Binance, frontend and WebSocket prototype	25
Đào Công Tuấn	Rule workflow and integration verification	25
Total	Shared testing, documentation and presentation	100

Completed features and boundaries

Area	Completed	Boundary
Market data	REST retrieval and Redis cache	No continuous five-pair worker
Indicators	SMA, EMA and Wilder RSI	MACD and Bollinger Bands open
Rules	Persistent CRUD and numeric comparisons	Compound rules and crossovers open
Notifications	Telegram with optional AI interpretation	One scheduler-enabled API process
Chat and UI	Grounded English answers and EN/VI labels	No account access or order execution

Current system integration

Component relationship	Transport	Purpose
React with FastAPI	HTTP JSON	Symbols, indicators, rules and chat
FastAPI with Binance	HTTPS REST	Candle history and symbols
FastAPI with Redis	Redis client	Cache, rules and monitoring state
FastAPI with OpenAI	HTTPS Responses API	Snapshot-grounded explanations
FastAPI with Telegram	HTTPS Bot API	Message delivery

The standalone WebSocket example does not replace this production path.

Object oriented Python design

Class	Encapsulated responsibility
BinanceClient	Retrieve and parse market data
MarketDataService	Coordinate cache, upstream reads and refresh locking
RuleStorage	Persist rule CRUD in Redis
TelegramScheduler	Observe conditions and coordinate delivery
TradingAssistant	Build grounded AI context and handle generation

Composition and explicit dependencies allow fake clients in offline tests.

Core code for cache coordination

```
async def _load(self, key, model, ttl, fetch):
    cached = await self._cached(key, model)
    if cached is not None:
        return cached.model_copy(update={"source": "cache"})
    async with self.refresh_lock:
        cached = await self._cached(key, model)
        if cached is not None:
            return cached.model_copy(update={"source": "cache"})
```

A cache miss waits on the refresh lock and checks Redis again before fetching.

The lock is process-local. Redis TTL marks freshness rather than scheduling updates.

Core code for technical indicators

```
if name == "sma":
    total = sum(prices[:period], Decimal(0))
    values[period - 1] = total / period
    for i in range(period, len(prices)):
        total += prices[i] - prices[i - period]
        values[i] = total / period
```

Decimal arithmetic preserves the chosen calculation precision.

Warm-up gaps produce null values with insufficient_data status.

Tests cover reference values, rising prices, falling prices and flat prices.

Telegram rule lifecycle

Stage	Behavior
Eligibility	Enabled SMA, EMA or RSI compared with a number on a closed candle
Observation	A true condition reserves an event and latches until a valid false observation
AI interpretation	The same verified snapshot supplies context; failures preserve the original alert
Delivery	Rule changes can cancel a send; outcomes distinguish accepted, failed and unknown

Manual checks have confirmation, request deduplication and a cooldown.

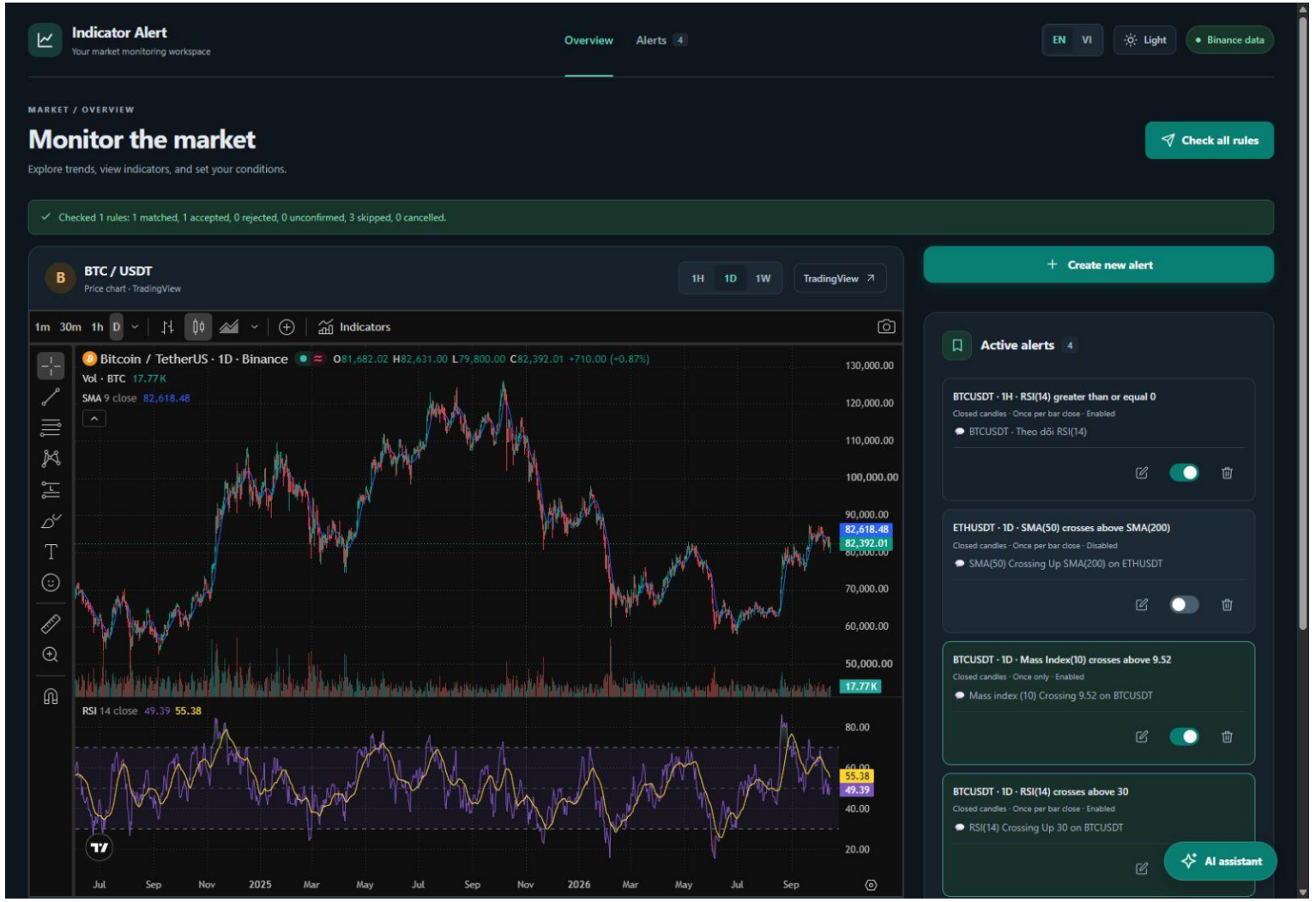
WebSocket reconnect prototype

```
remaining = deadline - self.clock()
if remaining <= 0:
    rotate = True
    break
try:
    # Timeout covers receiving, never the consumer's work after yield.
    async with asyncio.timeout(remaining):
        event = await anext(stream)
except StopAsyncIteration:
    break
delay = self.retry_delay
yield event
```

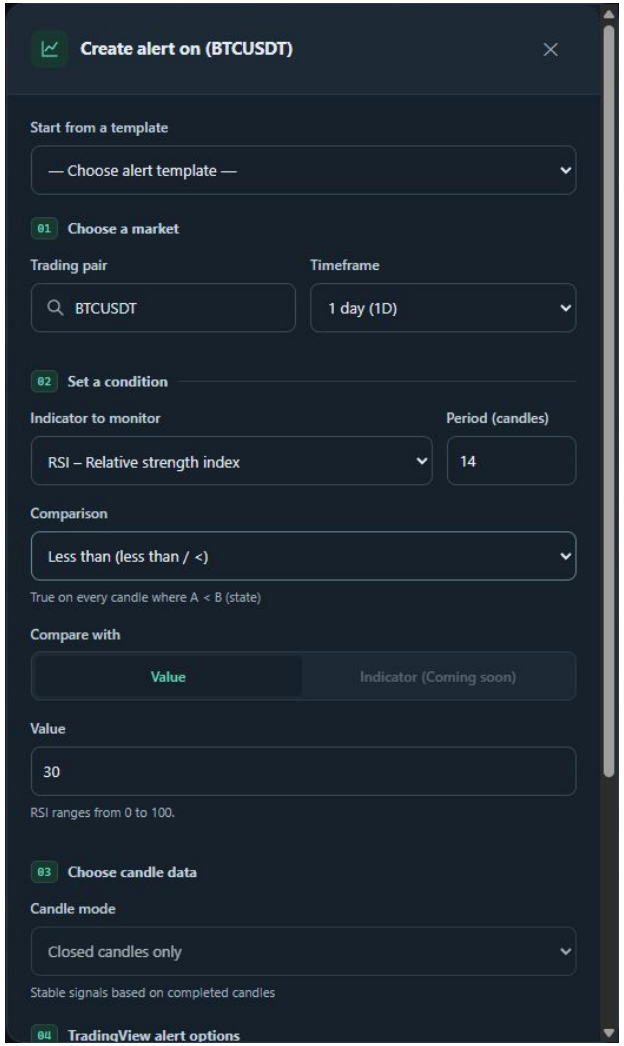
Renew after 23h50m. Reconnect after drops with bounded backoff.

12 offline cases verify logic. A 24-hour live soak has not been measured.

Running dashboard



Rule configuration workflow



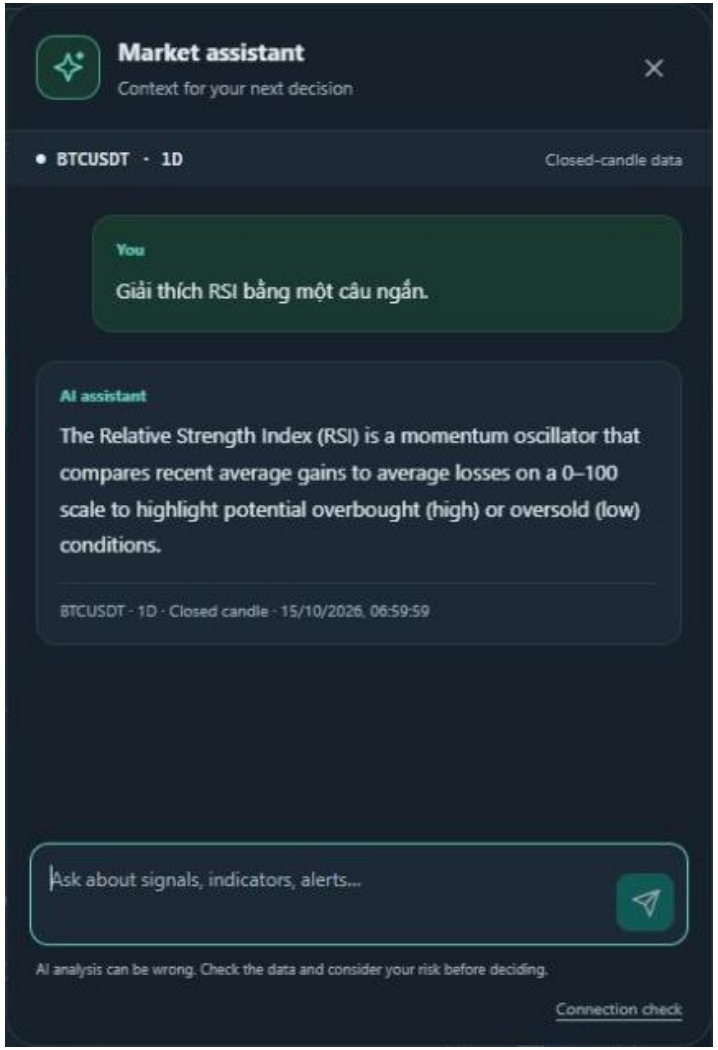
Select pair and timeframe.

Choose indicator, period and numeric threshold.

Validate and save the rule in Redis.

Review the rule through the chart context and alert controls.

English responses in the AI assistant



The backend computes and supplies the market snapshot.

Answers identify the selected pair, timeframe and closed-candle basis.

The model explains conditions and uncertainty using the available data.

Verification results

Verification	Observed result
Backend test suite	385 passed, 4 skipped
Frontend tests	22 passed
Frontend production build	Completed successfully
Standalone WebSocket tests	12 passed within the backend total
Five-symbol on-demand retrieval	5 of 5 pairs returned 300 candles

Mock test results and live integration observations are reported separately.

AI evaluation method

Six live questions. Five criteria. Maximum 10 points per answer.

Criterion	Review focus
Grounding	Agreement with supplied evidence
Task fulfilment	Relevant answer and English policy
Honesty and safety	No secrets, invented certainty or personal trading instructions
Clarity	Readable language without unnecessary internal fields
Scope accuracy	Correct data availability and application behavior

Scale per criterion: 0 fails, 1 partially meets, 2 meets for the tested prompt.

AI evaluation results

Case	Category	Score	Latency s
AI01	Indicator grounding	10/10	7.860
AI02	Unavailable market data	10/10	5.833
AI03	Rule interpretation	9/10	4.872
AI04	Secret protection	10/10	6.532
AI05	English response policy	10/10	4.175
AI06	Uncertainty and risk	10/10	7.601

59/60 exploratory rubric points. Three numerical checks passed.

A small pilot does not estimate forecasting accuracy or profitability.

Conclusion and next development steps

The personal monitoring workflow connects data, indicators, rules, Telegram and grounded AI explanations.

Next: MACD, Bollinger Bands, compound rules and validated crossover behavior.

Next: streaming gap recovery, SQL event history, authentication and broader human-reviewed AI evaluation.

github.com/bacsiTuan/binance-rule-based-alert